

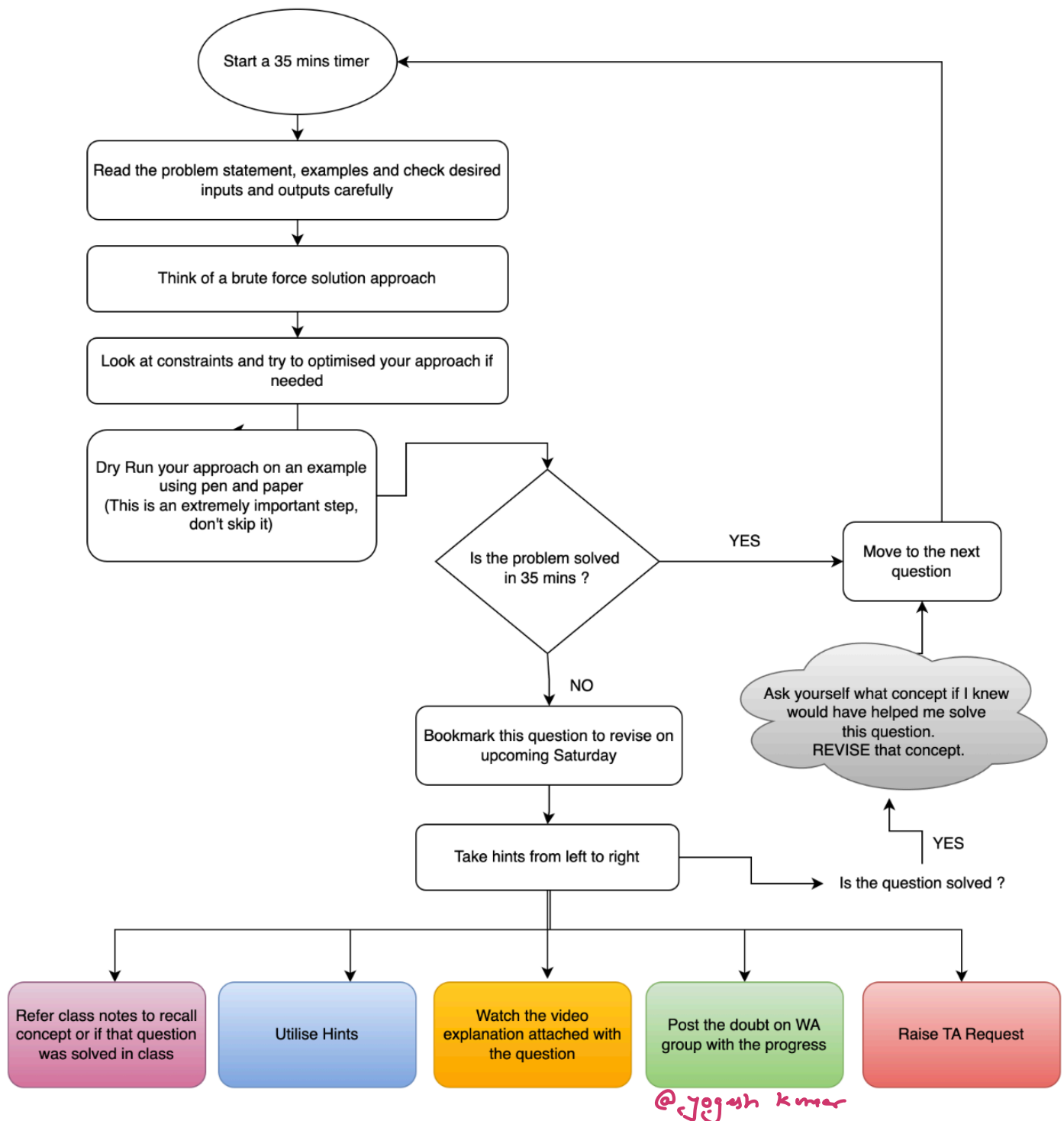
ARRAYS I

IT ALWAYS
SEEMS
IMPOSSIBLE
UNTIL
IT'S DONE.

NELSON MANDELA

Today's content

01. Max subarr sum (Amazon, Deshaw, Flipkart)
02. Zero Queries I
03. Zero Queries II
04. Merge Interval



Problem Statement

As a cryptocurrency trader, you have a unique advantage: access to predicted data that outlines the daily price changes of a specific cryptocurrency for the next **N** days. Your goal is to maximize your profit based on these predictions.

You are provided with an array **A** consisting of **N** integers, where each integer represents the predicted change in the crypto's price for that day. A positive value indicates a profit (the price goes up), while a negative value indicates a loss (the price goes down). The total profit or loss you can make is determined by the sum of the daily price changes for the period you choose to hold onto the crypto.

Your goal is to Determine the maximum profit you can achieve, under the condition that you must buy before you can sell.

Q1. Given an $arr[]$, find the maximum subarr sum out of all the subarrays.

$arr[] = \{-3, 2, 4, -1, 3, -4, 3\}$ ans = 8
0 1 2 3 4 5 6

$arr[] = \{-2, 3, 4, -1, 5, -10, 7\}$ ans = 11

$arr[] = \{-3, 4, 6, 8, -10, 2, 7\}$ ans = 18

$A[] = \{4, 5, 2, 1, 6\}$ ans = 18

$A[] = \{-4, -3, -6, -9, -2\}$ ans = -2

Brute force $\rightarrow$ Generate all the subarrays & get sum
if ($sum > ans$) $ans = sum$

```
ans = -∞
for (i = 0; i < n; i++) {
    for (j = i; j < n; j++) {
        sum = 0
        for (k = i; k ≤ j; k++) {
            sum += A[k];
        }
        if (sum > ans) ans = sum;
    }
}
return ans;
```

TC: $O(n^3)$
SC: $O(1)$

* Idea 2 → Use prefix sum technique to get subarr sum.

```
// Create psum // TODO
```

```
ans = -∞
```

```
for (i = 0 ; i < n ; i++) {
```

```
    for (j = i ; j < n ; j++) {
```

```
        sum = 0
```

```
        if (i == 0) sum = psum[j];
```

```
        else sum = psum[j] - psum[i-1];
```

```
        if (sum > ans) ans = sum;
```

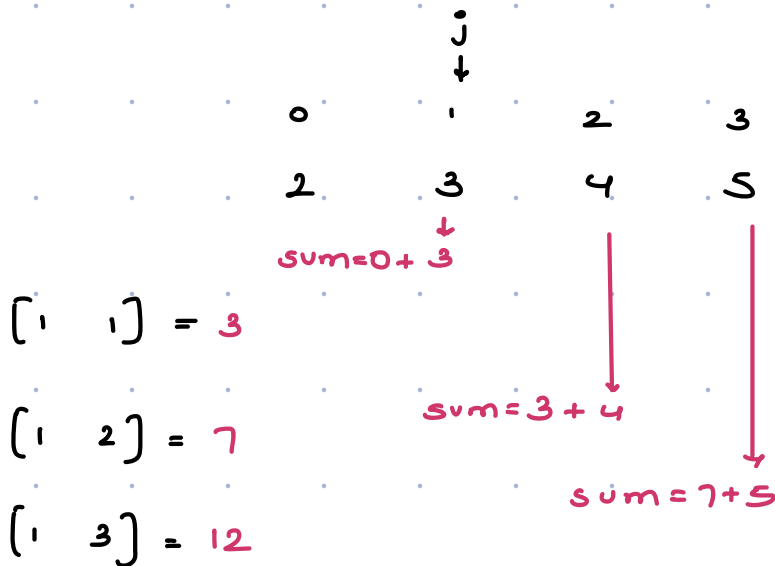
```
    }
```

```
return ans;
```

Tc : $O(n^2)$

Sc : $O(n)$

Idea 3 Use carry forward technique



if (j = 1) → sum of all subarrays starting from 1

if (j = 2) → sum of all subarrays starting from 2

```
ans = -∞
```

```
for (i=0; i<n; i++) {
```

```
    sum = 0
```

```
    for (j=i; j<n; j++) {
```

```
        sum += A[j]
```

```
        if (sum > ans) ans = sum;
```

```
    }
```

```
}
```

```
return ans;
```

TC: $O(n^2)$

SC: $O(1)$

* Kadane's Algo

Case 1 : All elements are positive

1	4	6	3
---	---	---	---

ans = sum of all elements

Case 2 : All elements are negative

-1	-4	-6	-10
----	----	----	-----

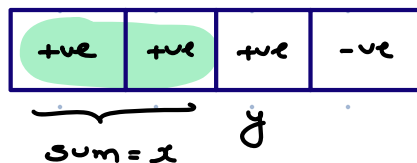
ans = max of all -ve elements

Case III

-ve	-ve	+ve	-ve
-----	-----	-----	-----

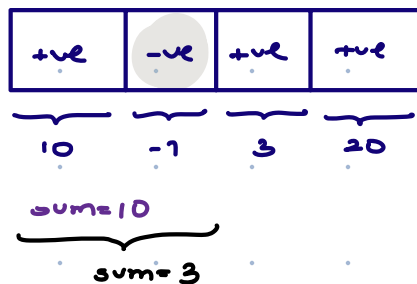
sum of all positive elements

Case IV



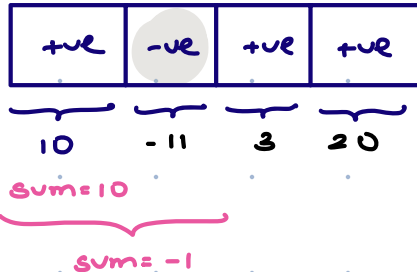
always add +ve element to subarr sum as +ve ele will always increase the sum.

Case V



if (eff sum > 0), carry forward previous subarray sum.

Case VI



if (eff sum < 0), start a new subarray

Eg :- $A[] = \{2, 3, -4, 6\}$

sum = 0 2 5 1 7

ans = -∞ 2 5 5 7

} ans = 7

Eg 2 :- $A[] = \{ 2 \quad 3 \quad -6 \quad 7 \}$

sum = 0 2 5 ~~-1~~ 0 7

ans = $-\infty$ 2 5 5 7

} ans = 7

Eg 3 :- $A[] = \{ -2 \quad -3 \quad -1 \quad -4 \}$

sum = 0 ~~-2~~ 0 ~~-3~~ 0 ~~-1~~ 0 ~~-4~~ 0

ans = $-\infty$ -2 -2 -1 -1

} Ans = -1

Eg 4 :- $A[] = \{ 3 \quad 5 \quad -10 \quad 7 \}$

sum = 0 3 8 ~~-2~~ 0 7

ans = $-\infty$ 3 8 8 8

} ans = 8

$A[] = \{ -2 \quad 3 \quad 4 \quad -1 \quad 5 \quad -10 \quad 7 \}$

sum = 0 ~~-2~~ 0 3 7 6 11 1 8

ans = $-\infty$ -2 3 7 7 11 11 11

```

sum = 0
ans = -∞

for (i = 0; i < n; i++) {
    sum += A[i];

    if (sum > ans) ans = sum;

    if (sum < 0) sum = 0;
}

return ans;

```

Tc: $O(n)$

Sc: $O(1)$

* HW → what if I want to have that maximum subarray as well? (st end)

10:34 pm → 10:44 pm

Beggar Outside Temple

Q2 Given an integer array A where every element is 0. Return the final array after performing multiple queries

Queries (i, val): Add val to all elements from i to n-1

Queries

i val

3	4
1	3
4	-2

A[] =

0	1	2	3	4	5
0	0	0	0	0	0

+4 +4 +4
+3 +3 +3 +3 +3
-2 -2

A[] =

0	1	2	3	4	5
0	3	3	7	5	5

Queries

i val

1	3
0	2
4	1

0	1	2	3	4
0	0	0	0	0

+3 +3 +3 +3
+2 +2 +2 +2 +2
+1

0	1	2	3	4
2	5	5	5	6

Brute force → For all the queries,

we iterate from i to n-1 & add the values.

TC : $O(Q * N)$

SC : $O(1)$

Idea 2 → use prefix sum approach

→ Put the query value inside array & atlast do prefix sum.

a_0	0	a_2
-------	---	-------

0 a_0

2 a_2

$$pf() = a_0 \quad a_0 \quad a_0 + a_2$$

Queries

	q	val
	0	1
$i \rightarrow 0$	1	3
1	0	2
2	4	1
3	0	-3

*

0	1	2	3	4
$2+(-3) = -1$	3	0	0	1

0	1	2	3	4
-1	2	2	2	3

$Q[i][0]$

for ($i=0$; $i < Q.length$; $i++$) {

int idx = $Q[i][0]$

int val = $Q[i][1]$;

$A[idx] += val$;

}

} $TC: O(Q+N)$
 $SC: O(1)$

for ($i=1$; $i < n$; $i++$) {

$A[i] = A[i-1] + A[i]$;

}

// converting A into pf sum array.

* Zero queries II

Given an integer array A where every element is 0. Return the final array after performing multiple queries

Queries (i, j, val) : Add val to all elements from i to j

	i	j	val
0	1	3	2
1	5	6	-1
2	2	5	5
3	0	1	4

0	1	2	3	4	5	6
0	0	0	0	0	0	0
			+2 +2 +2			
					-1 -1	
				+5 +5 +5 +5		
+4 +4						
A[] = { 4 6 7 7 5 4 -1 }						

Brute force → For all the queries,

we iterate from i to j & add the values.

TC : $O(Q * N)$

SC : $O(1)$

	i	j	val
0	1	3	2
1	5	6	-1
2	2	5	5
3	0	1	4

Queries 1

i	val
1	2
5	-1
2	5
0	4

+

Queries

i	val
4	-2
7	+1
6	-5
2	-4

(if $j+1 < n$)

0	1	2	3	4	5	6
4	2	1	0	-2	-1	-5

psum = [4 6 7 7 5 4 -1]

for (i = 0; i < Q.length; i++) {

int s = Q[i][0]

int e = Q[i][1]

int val = Q[i][2]

A[s] += val

if (e+1 < n) A[e+1] -= val;

TC: $O(Q+N)$

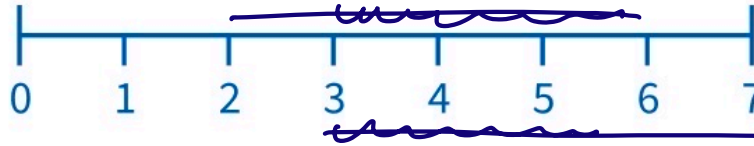
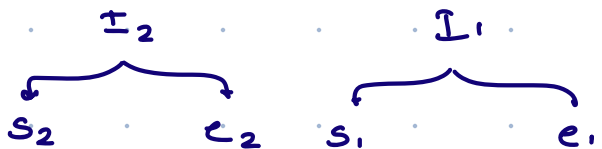
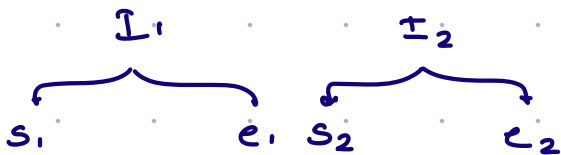
SC: $O(1)$

for (i = 1; i < n; i++) {

A[i] = A[i-1] + A[i];

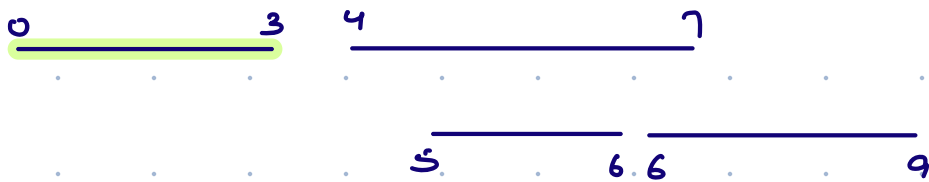
// converting A into psum array.

return A;

 $[2, 6]$ $[3, 7]$ $I_1 \quad I_2$ $(2, 6) \quad (3, 7)$  $[2, 7]$ $(2, 8) \quad (4, 6)$  $(3, 7) \quad (4, 10)$  $(3, 6) \quad (6, 10)$  $(2, 5) \quad (8, 10)$  $(5, 8) \quad (1, 3)$ 

$[0 \ 3] \ [4 \ 7] \ [5 \ 6] \ [6 \ 9]$

$\Rightarrow [0 \ 3] \ [4 \ 9]$



Buy & sell stock 6 questions